Measuring the Overhead of Trust Mechanisms in the X-Road Data Exchange Layer

Ruslan Nechshadimov
University of Tartu
Supervisor: Artjom Lind

Draft v0.2, Oktober 14, 2026

Abstract

WIP

1 Introduction

When one organisation acts on data received from another, it depends on something it did not produce and cannot check by itself. Trust is the name for that dependence. In organisational research it is defined as the willingness of a party to be vulnerable to the actions of another party, based on the expectation that the other will perform an action important to the trustor, without being able to monitor or control it [17]. In computing the same idea is phrased as a belief in the competence of an entity to act dependably, securely and reliably within a specified context [18]. In security engineering a *trusted* component is distinguished from a *trustworthy* one: the first is a component whose failure can break the security policy, the second one that will not fail [19]. A well-designed system therefore tries to need as little trust as possible, and to place the rest in a few components that are named explicitly and protected accordingly.

X-Road, the data exchange layer, is built on this principle. Its member organisations do not have to trust each other's systems directly. Their trust rests on a root of trust, a point that is accepted as trusted axiomatically [3]. In X-Road this root is centralised: it is formed by the certification authorities that vouch for the members' keys [3], together with the Central Server, which publishes who the members are and which authorities to trust, and the time-stamping authority [11]. Every message then carries evidence that can be checked against this root. It is signed by the sender's Security Server and verified by the receiver's, written into the message logs on both sides and time-stamped, and it travels over a mutually authenticated Transport Layer Security (TLS) channel, while the status of the certificates is checked through the Online Certificate Status Protocol (OCSP) [11, 1]. Trust in the other party is replaced by verification against the root of trust, and an exchange can later be used as evidence of who sent what and when. The other side of a centralised root is that it becomes a single point of failure [3], which is why alternatives to it are being studied.

Verification is not free. Signing, verification, logging and setting up the channel take computation and bytes, some of them on every message and some in the background. X-Road has been measured before, but end to end, for questions other than the cost of these mechanisms: a study of threshold signatures reports end-to-end round trips of about 32 ms with all components on one laptop and presents them as a lower bound [4]; a capacity study measures the overhead of a Security Server in front of one registry service, where database time dominates [7]; and the paper that introduced batch time-stamping gives an analytical estimate and leaves benchmarking of the implementation to future work [1]. The Nordic Institute for Interoperability Solutions (NIIS), which develops X-Road, does not publish performance data. None of these sources splits the cost of an exchange by mechanism.

Two developments make that missing split more relevant. The first is volume: the design of batch time-stamping was calibrated against about 240 million queries per year [1], while the Estonian instance now reports 3.74 billion [10], so whatever is paid per message is multiplied by a much larger base. The second is a set of proposals that would make the trust mechanisms themselves more expensive. Post-quantum signatures are expected to enlarge certificates, signatures and handshakes [5, 6]; a Dilithium5 signature, for example, takes 4595 bytes against the 256 bytes of RSA-2048 [6]. Decentralised Public Key Infrastructure (PKI) built on verifiable credentials adds a presentation to every request [2], and threshold signatures replace a local key with a distributed protocol [4]. These works describe what will become more expensive, but without a measured baseline for the present system they cannot say by how much.

The proposals above do not change X-Road as a whole but individual mechanisms. Post-quantum algorithms affect signing, verification and the TLS handshake [5], a decentralised PKI changes how certificates are checked [2], and threshold signatures change how messages are signed [4]. The cost of each change can only be judged against the cost of the mechanism it replaces, and since X-Road keeps certificate status checking and time-stamping out of the request path by design [1], it also matters which of these costs are paid on every message. Larger post-quantum signatures and certificates make messages bigger, and the growth in volume means more messages processed at the same time, so the costs have to be known as a function of message size and load. Finally, the most recent figures were obtained with all components on one machine, which their authors present as a lower bound [4], whereas the Security Servers of two organisations normally communicate over a network, and the NIIS report expects post-quantum handshakes to add round trips on it [5]. This leads to the following research questions:

RQ1
: How much do the trust mechanisms of an exchange — the connection between the Security Servers, signing and verification, message logging, time-stamping and certificate status checking — contribute to its latency and to the bytes on the wire, and which of these costs are paid on every message rather than deferred?

RQ2
: How do those contributions change with message size, and how does the throughput of each configuration change with the number of concurrent clients?

RQ3
: Does the answer change when the Security Servers run on separate machines instead of one host, and how does it change with a round-trip time added between them?

The contribution is a measurement rather than a new method: a testbed that can be redeployed on one or several hosts, the differences between configurations in milliseconds and bytes and the number of background requests from three environments, a descriptive per-stage breakdown taken from X-Road's own operational monitoring, the effect of a controlled added round-trip time together with a prediction made before that measurement, and the raw data. The study measures X-Road as it runs today, with its default keys: post-quantum schemes are not yet available in X-Road [5, 6], so their effect is discussed rather than measured.

2 Background: what is paid per message

X-Road is a centrally managed distributed data exchange layer. A Central Server distributes the global configuration — the member registry, the trusted Certification Authorities (CAs) and the trusted time-stamping authorities — and each member operates a Security Server that handles key management, signing, secure transport, logging and access control [2, 11].

For this study the mechanisms divide by their position relative to the request path:

- **Synchronous**, paid on every message: the mutually authenticated TLS channel between the Security Servers; signing with the member's signing key; verification of the incoming signature; and writing request and response into the message log, whose bodies are logged in full by default [12]. How much the TLS channel costs per message depends on whether connections are reused [13]. By default they are not: the client-side pool reuse (`pool-enable-connection-reuse`) and the provider-side support for pooled connections (`server-support-clients-pooled-connections`) are both off, so each message opens a new TCP connection and performs a TLS handshake [12].

- **Asynchronous**, amortised or deferred: certificate status checking, which uses cached OCSP responses refreshed on a fixed interval (20 minutes by default) rather than one query per message; time-stamping, applied in batches over a hash chain of log records (once per minute by default, with a tolerated time-stamping outage of four hours, the parameter `acceptable-timestamp-failure-period` being 14400 seconds); and the periodic refresh of the global configuration [9, 12, 1]. Because one time stamp covers a whole batch, its cost per message is divided by the size of the batch and falls as the load grows [11, 1].

The split is a documented design decision: caching status responses and deferring time stamps were introduced to keep a per-message PKI operation from becoming both a single point of failure and a latency source [1]. It carries a security price as well, since a cached, nonce-less OCSP response leaves a window during which a revoked certificate is still accepted [2]. The performance side of that trade-off is the subject of this study.

3 Methodology

3.1 Testbed

The testbed runs X-Road 7.8.3 in containers built from the official images: one Central Server, a test CA that also provides the OCSP and time-stamping services, and four Security Servers — a management server, one consumer (*ss1*) and two providers (*ss2*, *ss3*). The measured path is *load generator* → *ss1* → *ss2* → *echo service*; the second provider is used only to check what limits the consumer under load. The echo service answers a REST request with a body of the requested size, filled with random bytes so that the message log cannot compress it, and its own processing time is negligible. The load generator is k6, sending REST `GET` requests in a closed loop with a given number of concurrent clients, so the message size below is the size of the response body.

Every Security Server is limited to 2 CPU cores and 4 GB of memory and the Central Server to 2 GB, in all environments, so that the environments differ in where the containers run rather than in how much hardware each of them receives. The whole testbed, including member and service registration, is set up by scripts from a single composition file and can be redeployed from scratch.

3.2 Environments

Three machines. Three identical laptops (Intel Core i5-8250U, 4 cores and 8 threads, 16 GB of memory) connected by a dedicated wired switch. The first runs the Central Server, the test CA, the management server and the second provider, the second runs the consumer and the load generator, and the third runs the provider and the echo service. The containers have their own addresses on the switch network (Docker `ipvlan`), so the traffic between the Security Servers is not translated by the host and leaves the machine. Clocks are synchronised with chrony to within half a millisecond, which matters when timestamps from two machines are compared.

One machine. The same stack on one of these laptops alone. With the same CPU and the same container limits, the difference to the three-machine environment is the network between the Security Servers and the sharing of one CPU by all components.

Development laptop. The same stack on a newer laptop (Intel Core Ultra 7 265U) under Windows 11, with the containers running in WSL 2. It shows how the absolute numbers depend on the CPU.

Emulated network. In all of the environments above the Security Servers are at most one switch apart, with a round-trip time below a millisecond, whereas the Security Servers of two organisations usually communicate over a longer and less predictable path. From local measurements alone it is not possible to tell whether the results would hold there. A controlled round-trip time is therefore added between the consumer and the provider on the three machines with netem, the network emulator of the Linux kernel [16], which adds a chosen delay to the packets of a given flow. This is not a model of a real wide-area deployment, only of the added round-trip time. The same approach is used to study TLS under different network conditions without Internet-scale infrastructure [15], and the accuracy of the delay netem produces has been examined in [20, 21]. How the delay is applied and verified is described in Section 3.6.

On all machines the CPU runs in its performance power mode, which is checked before every grid and recorded with each run, since the power mode affects the latency.

3.3 Isolating the cost of a mechanism

The trust mechanisms of RQ1 cannot all be switched off one at a time, so each of them is observed in the way that changes as little else as possible (Table 1).

Mechanism	Observed through
The whole X-Road path	*direct* compared with *full*
Inter-server connection policy	*reuse* compared with *full*
Logging of message bodies	*nobody* compared with *full*
Signing and verification	stages of the breakdown (Section 3.4)
Time-stamping	requests to the time-stamping service; *synctsa* as another policy
Certificate status checking	requests to the OCSP responder

Table 1: How the cost of each trust mechanism is observed.

Where a setting removes or changes a mechanism, a configuration is compared with the default one (*full*) and differs from it in that setting only:

- **direct** — plain HTTP from the load generator to the echo service, without X-Road. Its difference to *full* is the whole X-Road path, not any single mechanism.
- **reuse** — connections between the Security Servers are kept open (`pool-enable-connection-reuse`, `server-support-clients-pooled-connections` and `server-connector-max-idle-time`). By default each message opens a new connection [12], so the difference is the effect of the connection policy between the Security Servers: setting up the TCP connection and the TLS session and starting every transfer on a new TCP connection [13]. It does not isolate the TLS handshake.
- **nobody** — message bodies are not written to the message log (`message-body-logging=false`), so the difference is the cost of logging the body.
- **synctsa** — every message is time-stamped synchronously (`timestamp-immediately=true`). This is a different policy rather than the cost of the default batched time-stamping, and is reported as such.

Signing and verification cannot be switched off, so they are observed only in the stages of the breakdown in which they happen, together with the other work done in the same stages, rather than as a difference between configurations.

The two background mechanisms cannot be moved into the request path without changing what is measured. Postponing time stamps by a long interval would violate a documented constraint between the time-stamping interval and the OCSP freshness [12]. Making the OCSP freshness shorter than the interval at which the signer refreshes its cached responses makes every request fail, because the Security Server takes certificate status only from that cache and does not query the responder while it processes a message. For these two mechanisms the cost is therefore observed as the number of requests sent to the time-stamping service and to the OCSP responder during each run and while its messages are being time-stamped.

3.4 What is measured

For each request the load generator records the latency and whether a correct response of the expected size arrived. For each run the bytes exchanged between consumer and provider, counted by `iptables` inside the network namespace of the consumer on the X-Road port, the growth of the message log, and the number of OCSP and time-stamping requests from the CA log are recorded.

X-Road also records four timestamps per request on each Security Server in its operational monitoring database: when the request arrived, when it was forwarded, when the response arrived and when it was returned. Joining the records of the consumer and the provider by the request identifier splits the latency into six consecutive stages: the consumer before sending, the way to the provider (the stage in which signing and logging on the consumer and setting up the connection take place), the provider before forwarding (the stage in which verification and logging take place), the echo service, the provider returning the response, and the consumer finishing it. The stages add up to the end-to-end time. The breakdown is descriptive: these are timestamps of the measured system itself with a resolution of 1 ms, and the share of requests whose records could be matched on both servers is reported with it. Stages measured within one host are more reliable than the two stages that span hosts, where the residual clock offset enters the result; explanations that involve the network are checked against packet captures (Section 3.6).

3.5 Procedure

Two message sizes are used with one client. Post-quantum signatures and certificates would add a few kilobytes to every message [6], which matters most for small messages, so a 10 KB response represents a small message; a 1 MB response represents a large one, whose body the message log stores in full by default [12]. One client with no other load gives the baseline latency without queueing, and the load is then increased until one of the components runs out of resources [7]: the number of clients grows beyond the two CPU cores available to each Security Server, to 4, 16 and 32, so as to reach the throughput ceiling of the Security Servers; this range covers the 10 to 30 concurrent clients used in [7]. The main grid therefore crosses the five configurations with 10 KB and 1 MB responses at one client and 10 KB at 4, 16 and 32 clients, plus the two-provider case at 4, 16 and 32 clients.

The Security Server runs on the Java virtual machine, whose first requests after a start are slower until the code is compiled, so only the steady state is measured [14]. After every configuration change the Security Servers are restarted and warmed up by a discarded run at four clients, followed by warm-up requests before every measured run. The amount of warm-up is set from a pilot run as the point after which the median latency over consecutive windows of requests stops decreasing. Each run measures 1000 requests, as in [4], and 200 at 1 MB, where every message adds about a megabyte to the message log of each Security Server.

Following [14], every point is measured in several independent runs rather than once. A grid repeats every point five times in random order, and a control run with the default configuration at the start of each repetition and at the end shows the drift of the testbed during the grid. Several grids are run and combined. During the measurements the machines run nothing else. Before the message log is cleaned, the runner waits until every logged message has its time stamp, so that deferred work is not lost between runs.

Results are reported as the median over repetitions of each run's median, with the interquartile range between repetitions and the number of successful requests.

3.6 Added round-trip time between the Security Servers

A controlled round-trip time is added between the consumer and the provider with netem [16], as done for TLS benchmarking in [15]. Half of the round-trip time is added on each side, in the network namespaces of the two Security Servers and only for packets to the other server, so that all traffic between them is delayed, including the OCSP exchange between Security Servers, while the client, the Central Server and the CA stay on the local network. The delay is fixed, without jitter, since netem reproduces jitter less accurately [21, 20]. The results are reported as the effect of a controlled added round-trip time between the Security Servers, not as a measurement of a real wide-area deployment. A control at zero delay with the same netem rules installed checks that the rules themselves do not change the result.

The added round-trip time (0, 5, 20 and 50 ms) is verified on the measured flow itself: during every run the round-trip time of the consumer's connections to the provider is read from the kernel. Before the grid, a pilot with traffic captures of sequential requests identifies for every setting whether a request opens a new connection, with a full or a resumed TLS handshake, or uses an existing one, and how many times one side waits for the other. From these exchanges a prediction is made of how latency should grow with the round-trip time, and it is compared with the measured slope. Captures of a 1 MB response with and without connection reuse check how the transfer of a large response depends on the state of the TCP connection.

The delay grid crosses *full* and *reuse* with 10 KB and 1 MB responses at one client and 0, 5, 20 and 50 ms, and an extended grid adds 100 KB responses and 4 and 16 clients at 0, 20 and 50 ms.

3.7 Threats to validity

Several properties of the testbed shift the absolute values. The machines are laptops rather than servers, and the development laptop adds the overhead of WSL 2 on Windows. The test CA, OCSP responder and time-stamping service run next to the Security Servers, whereas in production they are external services with their own latency, so the figures for status checking and synchronous time-stamping are optimistic. All intervals of the background mechanisms are left at their defaults, while a production deployment may use longer ones: for the Estonian instance of 2017 a master's thesis documents a global configuration validity of six hours and an OCSP freshness of eight hours [8], so background costs per message would be even lower there than in the testbed.

Other properties change the shares and the spread rather than the absolute values. The echo service has no processing time of its own, which raises the relative share of the trust mechanisms compared with a real service. Under load the spread between repetitions is larger, so there the stable throughput and the results of synchronous time-stamping are reported together with the spread and the errors, and small differences between the other configurations are not assigned to individual mechanisms. The added round-trip time is emulated: it has no cross traffic and no losses.

Because of these factors the absolute latencies describe this testbed rather than a production deployment. The comparisons are therefore made between configurations on the same machines

and under the same conditions, and the results are read primarily as differences between configurations and as their change with message size, load and added round-trip time; absolute values are reported for reference only.

4 Results

4.1 Stability of the testbed

The warm-up needed turned out much longer than expected. The closest measurement of X-Road reports means over 1000 requests without describing a warm-up [4], and a first short check on the testbed suggested that about 700 requests would be enough. A longer pilot showed otherwise: the median latency over windows of 200 requests after a restart of the Security Servers fell from 114 to 45 ms and reached a plateau only after 2500 to 3000 requests, and the first run after a restart remained slower even after 4000 requests at one client. Repeated runs at the same point differed by up to 20 %. These observations set the warm-up and the number of repetitions described in Section 3.5. The effect of the power mode of the CPU was found in the first measurements on three machines. The default configuration took 78 ms with one client, against 41 ms on one of the same laptops alone, and the difference looked like a cost of the network. Changing how the Security Servers acknowledge TCP segments and how the load generator reuses its connection did not change it. The per-stage timestamps then showed all stages, including purely computational ones, 2 to 2.5 times slower than on the development laptop (for example, signing the response on the provider took 22 instead of 10 ms), so the computation had slowed down, not the network. The laptops were in an energy-saving mode, in which an almost idle CPU does not raise its frequency for short bursts of work; on one of them the control run took 79.7 ms in that mode and 28.7 ms in the performance mode. All results below were obtained in the performance mode; the earlier runs in the energy-saving mode are not used.

With five repetitions per grid, the main grid was repeated 30 times on three machines (six grids), 15 times on one machine (three grids) and 20 times on the development laptop (four grids). The control run (*full*, 10 KB response, one client) had a median of 28.7 ms over 36 runs on three machines (27.5 to 37.2 ms, one outlier), 30.4 ms over 18 runs on one machine (28.9 to 31.2 ms) and 26.7 ms over 24 runs on the development laptop (24.3 to 27.9 ms). Every request succeeded except in *reuse* under load, where single requests failed with HTTP 500 in some runs: 7 of 90 000 requests at 4 to 32 clients on three machines, 3 of 45 000 on one machine and 2 of 60 000 on the development laptop. The failures are consistent with a pooled connection that was already closed by the other side.

4.2 Cost of the mechanisms (RQ1)

Table 2 shows the latency with one client. The difference between *direct* and *full* is the whole X-Road path: on three machines 27.6 ms for a 10 KB response and 64 ms for a 1 MB response. Logging the message body makes no difference beyond the spread at 10 KB, but at 1 MB not logging it saves 34.9 ms, 47 % of the latency. Keeping connections open between the Security Servers saves 2.0 ms at 10 KB and 4.3 ms at 1 MB. Synchronous time-stamping adds 89–92 ms. The interquartile range between repetitions is 0.7–4.8 % of the median.

On three machines a 10 KB exchange carries 14.8 KB from consumer to provider and 25.7 KB back; with *reuse* 11.5 and 22.8 KB, so the connection policy costs 3.3 and 2.9 KB per message in the two directions. The message log of the consumer grows by 33.7 KB per exchange with bodies and 19.3 KB without them, and at 1 MB by 1.42 MB and 19.5 KB.

The two background mechanisms did not appear in the request path. No run sent a single request to the OCSP responder, and a configuration with an OCSP freshness shorter than the refresh of the cached responses made every request fail, as described in Section 3.3. Batched time-stamping sent no requests to the time-stamping service during the runs and about two per

	Three machines		One machine		Development laptop	
	10 KB	1 MB	10 KB	1 MB	10 KB	1 MB
direct	0.7	10.0	0.1	0.7	0.1	0.7
full	28.3	74.0	29.8	74.7	24.8	49.7
nobody − *full*	−0.9	−34.9	−0.3	−32.0	−0.9	−17.0
reuse − *full*	−2.0	−4.3	−2.0	−4.3	−1.6	−2.9
synctsa − *full*	+88.9	+91.7	+91.8	+98.1	+62.4	+68.0
repetitions	30		15		20	

Table 2: Median latency with one client (ms) by response size and environment; differences to *full* in the same environment. Every request succeeded.

run, of about 1200 messages, while the runner waited for the time stamps; *synctsa* sent 4802 per run, four for each exchange, one for every record written to the message log.

Table 3 shows the per-stage breakdown on three machines. All requests of these runs were matched between consumer and provider. The stage before the consumer sends the request takes 0.1–0.2 ms, so the consumer records the request as forwarded when it starts sending, and signing and logging of the request fall into the stage on the way to the provider. The configurations change the stages in which their mechanism works: *reuse* changes only the way to the provider (−2.4 ms); not logging a 1 MB body shortens the provider's response by 16.9 ms and the consumer's finishing stage by 17.0 ms, that is, logging the body costs about 17 ms on each side; and synchronous time-stamping adds 20–25 ms to each of the four stages in which a record is logged. The two stages that span machines include the residual clock offset of below half a millisecond.

	Consumer sending	To provider	Provider forwarding	Echo	Provider response	Consumer finishing	Total
full, 10 KB	0.1	10.7	4.2	0.2	6.9	7.1	29.4
nobody − *full*	0.1	0.3	0.0	0.0	−0.8	−0.7	−1.2
reuse − *full*	0.0	−2.4	0.2	0.0	0.0	0.2	−2.1
synctsa − *full*	0.1	24.3	20.6	0.2	19.8	25.1	90.5
full, 1 MB	0.2	12.4	5.4	0.3	33.0	24.5	76.5
nobody − *full*	−0.1	−1.4	−1.2	−0.1	−16.9	−17.0	−37.3
reuse − *full*	0.0	−3.0	0.0	−0.1	−1.1	0.0	−3.4
synctsa − *full*	0.1	24.5	20.9	0.2	21.8	25.0	94.5

Table 3: Per-stage breakdown from operational monitoring on three machines, one client (mean of each run, median over 15 repetitions, ms; resolution 1 ms). "To provider" and "Consumer finishing" span two machines.

4.3 Response size and load (RQ2)

From a 10 KB to a 1 MB response, the latency of *full* on three machines grows from 28.3 to 74.0 ms, while *direct* grows from 0.7 to 10.0 ms. Without body logging the growth is from 27.4 to 39.1 ms, so most of the additional time at 1 MB is writing the body to the message log. The effect of the connection policy grows from 2.0 to 4.3 ms, and the cost of synchronous time-stamping stays at about 90 ms.

Under load the spread between repetitions grows to 15–41 % of the throughput for *full*, *nobody*, *reuse* and the two-provider case, compared with 7–10 % for *synctsa*. Table 4 therefore lists the highest throughput each configuration reached. The differences between *full*, *nobody*, *reuse* and two providers are within the spread and are not assigned to individual mechanisms. Synchronous time-stamping halves the ceiling on three machines (50 against 106 requests per

second). A second provider does not raise the ceiling either: with two providers the stages of each provider become shorter, but the consumer's finishing stage becomes longer, so the consumer is the limit.

	full	*nobody*	*reuse*	two providers	*synctsa*
Three machines	106	106	108	111	50
One machine	75	76	79	82	28
Development laptop	112	107	118	109	41

Table 4: Highest median throughput reached at 4, 16 or 32 clients, 10 KB response (requests per second).

4.4 Environment and added round-trip time (RQ3)

With one client, three machines and one machine give the same picture (Table 2): the end-to-end latency differs by 0.7–1.5 ms and the differences between configurations by at most 6.4 ms. *direct* shows that the switch adds 0.6 ms at 10 KB and 9.3 ms at 1 MB, but through X-Road this is offset by the provider having a CPU of its own: on three machines the provider's stages are shorter (4.2 against 5.7 ms and 6.9 against 9.0 ms), while the way to the provider is 0.6 ms longer. Under load three machines reach 1.4 to 1.8 times higher throughput (Table 4), because the work is spread over three CPUs. On the development laptop all absolute values are lower and the CPU-bound costs smaller (logging a 1 MB body 17 instead of 35 ms, synchronous time-stamping 62 instead of 89 ms), while the differences keep their sign and order.

Added round-trip time. The round-trip time measured on the connections between the Security Servers was 0.5, 6.1, 21.4 and 51.5 ms for an added 0, 5, 20 and 50 ms (smoothed by the kernel, which includes delayed acknowledgements); for new connections the minimum was 0.53, 5.54, 20.78 and 51.11 ms. With the netem rules installed and no delay, the latency did not change at 10 KB (31.0 against 30.6 ms for *full*, 27.7 against 28.9 ms for *reuse*); at 1 MB it was 3–4 ms higher (about 5 %, with overlapping ranges), two orders of magnitude below the effect of the added round-trip time.

Before the first grid with added delay, traffic captures on the local network showed that every message of *full* opens a new TCP connection and resumes the TLS 1.2 session from a session ticket, without exchanging certificates. Counting the TCP handshake, the TLS handshake and the request itself, a message of *full* waits for three round trips between the Security Servers and a message of *reuse* for one. The growth of latency is therefore described by its slope against the added round-trip time: a slope of 3 means that the latency grows by three times the added round-trip time, that is, every message waits for three round trips. The prediction was a slope of 3 for *full* and 1 for *reuse*. Figure 1 shows the measurement: at 10 KB the slope is 5.1 for *full* and 2.1 for *reuse*, so the prediction underestimated both. Captures under delay showed two waits it did not include: the last part of the request (842 bytes after about 10 KB) is held until the provider acknowledges the earlier data, which adds one round trip in both configurations, and on a new connection the response of about 26 KB does not fit into the initial TCP window and needs a second flight, which adds one more round trip to *full*. With these, the count is 5 and 2; a calibration with sequential requests on the three machines gave slopes of 4.7 and 1.9. A short wait inside the TLS handshake, between the message that switches the consumer to the negotiated keys and its first encrypted message, appeared at 20 ms and not at 50 ms and causes a variation of about 0.3–0.4 in the slope. The stage breakdown agrees: the slope of the way to the provider is 2.6 for *full* and 0.5 for *reuse*, and that of the provider's forwarding stage is 1 in both.

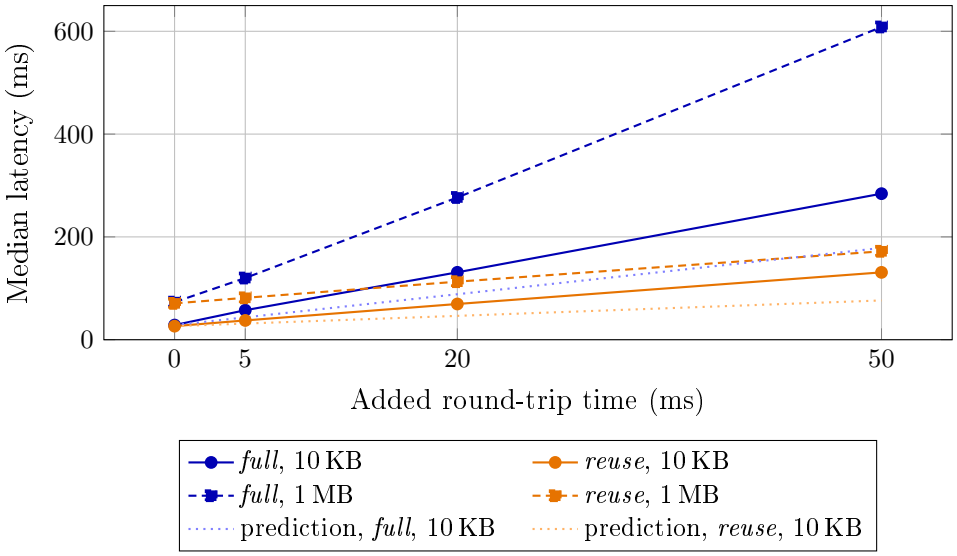


Figure 1: Latency with one client against the round-trip time added between the Security Servers, three machines, 20 repetitions (interquartile range 0.3–1.4 % at 20 ms and more); lines without marks show the prediction made before the measurement.

At 1 MB the slope is 10.7 for *full* and 2.0 for *reuse*. Captures of a 1 MB response at 50 ms show the reason: on a new connection the body leaves in about six flights, one per round trip, each about twice as large as the previous one (28, 57, 113, 226 and 453 KB), as the TCP window grows from its initial size; with *reuse* the window is kept between messages, and after the first request the whole response leaves in one flight. The effect of the connection policy thus grows from 2 ms on the local network to 153 ms (54 %) at 10 KB and 436 ms (72 %) at 1 MB with an added 50 ms.

The extended grid confirms the pattern. With a 100 KB response the slope is 7.0 for *full* and 2.1 for *reuse*, so with a new connection for every message the cost of distance grows with the size of the response (slopes of 5.0, 7.0 and 9.7 for 10 KB, 100 KB and 1 MB in this grid), while with reused connections it does not. With an added round-trip time the latency with four clients is the same as with one, and at 50 ms throughput is limited by the round trips: 14 requests per second for *full* against 30 for *reuse*. With 16 clients at 0 and 20 ms the Security Servers are the limit (103–127 requests per second) and the added round-trip time disappears in the queue (142 against 140 ms for *full*); at 50 ms *full* falls to 57 requests per second while *reuse* keeps 104.

TODO: to add in Discussion

Working list for the draft, to be removed before submission.

- Absolute values depend on the CPU while the differences between configurations hold across machines.
- What the measured costs mean for post-quantum signatures, a decentralised PKI and threshold signatures; the connection policy as the main cost once there is distance between the servers.

References

[1] A. Ansper, A. Buldas, M. Freudenthal, J. Willemson. High-Performance Qualified Digital Signatures for X-Road. In *NordSec 2013*, LNCS 8208, pp. 123–138, Springer, 2013.

[2] M. Bakhtina, K. L. Leung, R. Matulevičius, A. Awad, P. Švenda. A Decentralised Public Key Infrastructure for X-Road. In *ARES 2023*, ACM. DOI: 10.1145/3600160.3605092.

[3] M. Bakhtina, R. Matulevičius, A. Awad, P. Kivimäki. On the Shift to Decentralised Identity Management in Distributed Data Exchange Systems. In *SAC '23*, pp. 864–873, ACM, 2023. DOI: 10.1145/3555776.3577678.

[4] M. Bakhtina, J. Kvapil, P. Švenda, R. Matulevičius. Securing Organisational Digital Identity: A Full-Lifecycle Approach to Distributed Key Management. *International Journal of Information Security*, 25:131, 2026. DOI: 10.1007/s10207-026-01291-5.

[5] M. Bakhtina. Migration of X-Road to Post-Quantum Cryptography. NIIS Publication series, Nordic Institute for Interoperability Solutions, 2026. ISBN 978-9916-9853-4-2.

[6] N. Snetkov, J. Vakarjuk. Integrating Post-Quantum Cryptography to UXP. Cybernetica Research Report D-2-499, 2023.

[7] E. Taklai. X-tee teenuste jõudlustestimine Eesti äriregistri näitel (Performance testing of X-Road services on the example of the Estonian Business Register). Diploma thesis, Tallinn University of Technology, 2020.

[8] L. Mironova. X-tee platvormi kriitiliste kesksete teenuste testimine (Testing of the critical central services of the X-Road platform). Master's thesis, Tallinn University of Technology, 2017.

[9] P. Kivimäki. Resisting Failure. NIIS blog, 20 November 2020. `https://www.niis.org/blog/2020/11/20/resisting-failure`.

[10] Estonian Information System Authority (RIA). X-tee Factsheet, Estonia. `https://www.x-tee.ee/factsheets/EE/`. Figures based on data of 1 September 2026; accessed 17 September 2026.

[11] Nordic Institute for Interoperability Solutions. X-Road: Security Server Architecture. `https://docs.x-road.global/Architecture/arc-ss_x-road_security_server_architecture.html`. Accessed 17 September 2026.

[12] Nordic Institute for Interoperability Solutions. X-Road: System Parameters (UG-SYSPAR). `https://docs.x-road.global/Manuals/ug-syspar_x-road_v6_system_parameters.html`. Accessed 17 September 2026.

[13] D. Naylor, A. Finamore, I. Leontiadis, Y. Grunenberger, M. Mellia, M. Munafò, K. Papagiannaki, P. Steenkiste. The Cost of the "S" in HTTPS. In *CoNEXT '14*, pp. 133–140, ACM, 2014. DOI: 10.1145/2674005.2674991.

[14] A. Georges, D. Buytaert, L. Eeckhout. Statistically Rigorous Java Performance Evaluation. In *OOPSLA '07*, pp. 57–76, ACM, 2007. DOI: 10.1145/1297027.1297033.

[15] C. Paquin, D. Stebila, G. Tamvada. Benchmarking Post-quantum Cryptography in TLS. In *PQCrypto 2020*, LNCS vol. 12100, pp. 72–91, Springer, 2020. Cryptology ePrint Archive 2019/1447.

[16] S. Hemminger. Network Emulation with NetEm. In *linux.conf.au 2005*, Canberra, 2005.

[17] R. C. Mayer, J. H. Davis, F. D. Schoorman. An Integrative Model of Organizational Trust. *Academy of Management Review*, 20(3):709–734, 1995.

[18] T. Grandison, M. Sloman. A Survey of Trust in Internet Applications. *IEEE Communications Surveys & Tutorials*, 3(4):2–16, 2000.

[19] R. Anderson. *Security Engineering: A Guide to Building Dependable Distributed Systems*, 3rd ed. Wiley, 2020.

[20] L. Nussbaum, O. Richard. A Comparative Study of Network Link Emulators. In *Communications and Networking Simulation Symposium (CNS '09)*, 2009.

[21] A. Jurgelionis, J.-P. Laulajainen, M. Hirvonen, A. I. Wang. An Empirical Study of NetEm Network Emulation Functionalities. In *ICCCN 2011*, IEEE, 2011.